

Pointers Problem Set

CpE 1102L | Programming Logic and Design | Unit 8: Pointers

Problem 1: Pointer Arithmetic Trace

Predict the exact output of the program below, line by line. Then compile and run it to check your answers.

```
#include <stdio.h>
int main(void) {
    int arr[6] = {10, 20, 30, 40, 50, 60};
    int *p = arr + 1;
    int *q = &arr[4];

    printf("%d\n", *p + *q);
    printf("%d\n", *(p + 2));
    printf("%d\n", q - p);
    printf("%d\n", *p++);
    printf("%d\n", *p);
    return 0;
}
```

Problem 2: Pass by Reference, Fix the Bug

The function below is supposed to double every element of an array in place, but it will not compile as written. Identify the bug, explain why it happens, and rewrite the function so it works correctly.

```
void doubleArray(int arr[], int n) {
    for (int i = 0; i < n; i++) {
        int *p = arr[i];
        *p = *p * 2;
    }
}
```

Problem 3: Pointer to Pointer

Write a function with the prototype below. It should dynamically allocate an array of n ints using malloc, set every element to value, and make the caller's pointer point to the new array.

```
void allocateAndFill(int **p, int n, int value);
```

Then write a main function that:

- Declares an int pointer and calls allocateAndFill to build an array of 6 ints, all set to 7.
- Prints every element of the array using a loop.
- Frees the allocated memory before the program ends.

Problem 4: Array of Pointers with Dynamic Allocation

Write a complete program that does the following:

- a) Declares `int *arr[5]`, an array of 5 int pointers.
- b) Uses `malloc` to dynamically allocate one int for each of the 5 slots.
- c) Stores the values 100, 200, 300, 400, and 500 into the allocated memory, one value per slot.
- d) Prints all 5 values using pointer dereferencing, not array indexing on the values themselves (that is, use `*arr[i]` in the print loop).
- e) Frees every allocated block before the program returns.

Bonus: Add a second loop that prints the memory address stored in each slot using `%p`, so you can confirm the five ints live at five separate locations.

Problem 5: Pointer Compatibility and Multi-Level Indirection

Consider the following declarations and statements:

```
int a = 5;
int *pa = &a;
int **ppa = &pa;
int ***pppa = &ppa;

**ppa = 20;
*pa = *pa +>(*ppa);
***pppa = **ppa + 1;
```

- a) Draw a memory diagram showing `a`, `pa`, `ppa`, and `pppa`, and how each one points to the next.
- b) Trace the three statements in order and give the final value of `a`.
- c) Explain, in your own words, why `int *pa = &a;` is valid but `int **ppa = &a;` would be a compile error.
- d) Write one line of code that changes the value of `a` to 99 using only the name `pppa` (no direct use of `a` or `pa`).

Show all work. For trace problems, write the final printed values clearly. For code problems, submit compilable `.c` files.